SOCIAL EAGLE
learn. build. operate

WEEK 2 · DAY 4

Build a Multi-PDF RAG App with Sources

Assignment | learn. build. operate

1. Problem Statement

Build a **LangChain** and **Streamlit** RAG app that answers questions across **several PDFs at once**. The user uploads a folder's worth of documents; you chunk them, embed them and put them all in one **vector database**. Every answer must come back with its **source**: which file, which page, so the reader can go and check it.

Tools: Python, LangChain, Streamlit, FAISS, an OpenAI API key

2. Requirements

- Keep all your code in a single file named `rag_app.py`, run with `streamlit run`.
- Accept **multiple PDFs** in one upload, not one at a time.
- Chunk and embed all of them into a **single vector database**.
- Report how many **chunks were indexed** once building finishes.
- Answer questions using retrieval across every uploaded document.
- Show the **source of every answer**: file name and page number, pulled from the chunk metadata.
- Keep the store in `st.session_state`, and read your API key from an environment variable.

> **Think it through**
>
> With several documents in one database the honest answer is sometimes "these two disagree", how will your app show that rather than quietly picking one? And look at the page numbers your sources report: if a single answer cites five different pages, is your chunk size doing its job?

3. Steps to complete

These are hints, not the full answer. Fill in the blanks yourself.

Step 1: Set up the project

```
mkdir multi-rag && cd multi-rag
python -m venv venv && source venv/bin/activate
pip install streamlit langchain langchain-openai langchain-community \
            langchain-text-splitters pypdf faiss-cpu
export OPENAI_API_KEY="your-key"
```

Step 2: Index every upload into one store

`PyPDFLoader` already puts the page number in each document's metadata, carry that through the splitter and it survives into your results.

```
import streamlit as st
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS

files = st.file_uploader("Upload PDFs", type="pdf",
                         accept_multiple_files=True)

if files and "store" not in st.session_state:
    chunks = []
    for f in files:
        # save f, load with PyPDFLoader, split, add to chunks
        ...
    st.session_state.store = FAISS.from_documents(chunks,
                                                  OpenAIEmbeddings())
    st.success(f"Indexed {len(chunks)} chunks")
```

Step 3: Answer with sources

Retrieve the matching chunks, answer from them, then list where each came from.

```
docs = st.session_state.store.as_retriever().invoke(question)
# answer from docs, then show the metadata behind it
for d in docs:
    st.caption(f"{d.metadata['source']} - page {d.metadata['page']}")
```

4. Final output

```
Uploaded: policy.pdf, handbook.pdf, faq.pdf
Indexed 412 chunks.

Q: How many leave days do new joiners get?
A: New joiners get 12 days in their first year.
   Sources: handbook.pdf - page 7 | policy.pdf - page 2
```

5. What to submit

- A screenshot of an answer with its sources listed underneath.
- Your `rag_app.py` on **GitHub**, with the link shared in **WhatsApp**.
- Your screen-recorded video, posted in the **community**.